

Instruktioner för MA2

Det här dokumentet innehåller material för den andra modulen (MA2) i kursen Programmeringsteknik II. Modulen utgör en introduktion till datastrukturer i Python. Vi behandlar också klasser och några andra programkonstruktioner, som *operatoröverlagring*, *iteratörer* och *generatorer*.

Övningarna som hör till modulen finns i den här sortens gula rutor i texten. Det finns **14 obligatoriska övningar** totalt i modulen. Efter teori och ett antal exempel följer en grupp övningar på samma tema.

I denna modul kan det vara en bra idé att skriva egna klasser från grunden.

Modulen innehåller följande filer:

1. `linked_list.py` där du ska skriva lösningar på övningarna om länkade listor,
2. `bst.py` där du ska skriva lösningar på övningarna om binära träd,
3. testfiler med namn som `test_LinkedList_namn` och `test_BST_namn`, där `namn` är namnet på den klass som testas, som `LinkedList` och `BST`.
4. Filen `tests.sh` kan användas för att köra alla test från terminalen på vissa operativsystem.

Följ dessa instruktioner när du förbereder dig för att presentera dina lösningar:

1. Uppgifterna ska lösas i den ordning de finns i texten. Du ska använda de verktyg (klasser, funktioner, metoder, ...) som tas upp i övningen. Syftet är inte bara att få rätt resultat, utan att öva specifika tekniker.
2. Om inget annat anges ska du inte använda några ytterligare paket eller färdig kod, utöver det som finns i de filer du har laddat ned med uppgiften.
3. Alla funktioner måste namnges, ha argument och returvärden på precis det sätt som anges i uppgiften.
4. Din kod ska inte använda globala variabler.
5. Uppgifter som ska besvaras med text skrivs som kommentarer eller textsträngar sist i koddokumentet. Textsvaren kan hållas korta.
6. Redovisa dina lösningar för en lärare eller assistent i klassrummet eller ett Zoom-rum. Var beredd att svara på frågor om metodernas komplexitet.
7. När dina lösningar är muntligt godkända enligt punkten ovan:
 - (a) Fyll i ditt namn, e-post, namn på assistent/lärare som granskat uppgiften och datum för granskningen högst upp i dokumenten.
 - (b) Gör en zip-fil med `linked_list.py` och `bst.py` samt testfilerna och ladda upp den i STUDIUM.

Viktigt!

Du får samarbeta med andra studenter, söka på nätet och orientera dig med AI-verktyg, men du måste i slutänden skriva och kunna förklara din egen kod. Du får inte kopiera eller skriva av kod vare sig från andra studenter, automatiserade verktyg, eller från nätet förutom från de ställen som uttryckligen pekas ut i denna lektion.

Att byta variabelnamn och liknande ytliga redigeringar räknas inte som att skriva sin egen kod. Att hjälpa någon annan att fуска (om du exempelvis delar med dig av din kod till någon som försöker lämna in den som sin egen) är inte tillåtet och kommer att anmälas.

Eftersom uppgifterna ingår som moment i examinationen är vi skyldiga att anmäla markanta brott mot dessa regler till univeristets disciplinnämnd.

Allt kursmaterial skyddas av upphovsrätten och ska inte spridas vidare.

1 Datastrukturer

En *datastruktur* är ett sätt att organisera en samling av värden. Exempel på sådana samlingar är ett bilregister, en beskrivning av släktskapsrelationer, en tabell med avstånd mellan olika orter etc. De individuella objekten (alltså bilar, personer, orter, ...) kallar vi för *poster*.

I samband med datastrukturerna diskuterar man också vilka operationer man vill göra (effektivt) på samlingen. Exempel på operationer är:

- Lägga in nya poster.
- Ta bort poster.
- Söka poster med visst innehåll.
- Besöka alla poster.

Några vanliga datastrukturer:

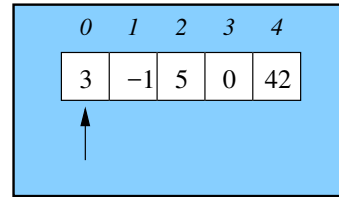
- Arrayer.
- Linjära länkade listor.
- Träd.
- Generella länkade strukturer ("grafer").
- Hash tables.

Arrayer

Detta är den mest fundamentala datastrukturen beroende på att den är nära kopplad till datorernas minnesorganisation. Typiskt för arrayer är att

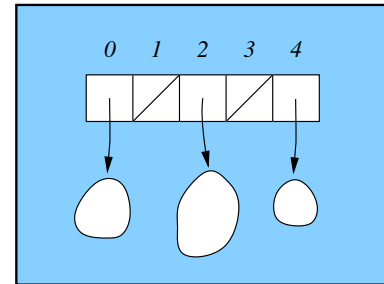
Typiska egenskaper hos arrayer:

- alla element är av samma datatyp (t.ex. integer) vilket innebär att de tar lika stor plats och
- alla element lagras i en sammanhängande minnesarea.



Dessa egenskaper gör att adressen till vilket element som helst lätt kan beräknas om man känner adressen till det första elementet (på plats 0) och elementstorleken.

Om elementen är olika stora kan man lagra pekare till elementen i arrayen (vilket ger en array av pekare):

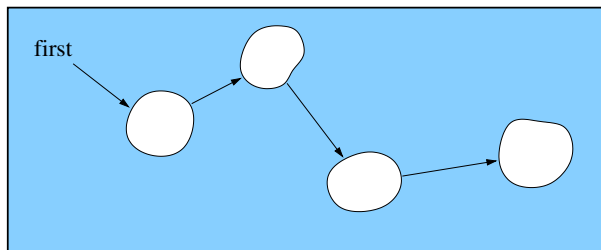


Den stora fördelen med arrayer är snabb index-operation ($\Theta(1)$).

En nackdel med arrayer är att det är besvärligare att lägga in ett nytt element på en given plats eftersom alla efterkommande element måste flyttas ($\Theta(n)$).

Linjära länkade listor

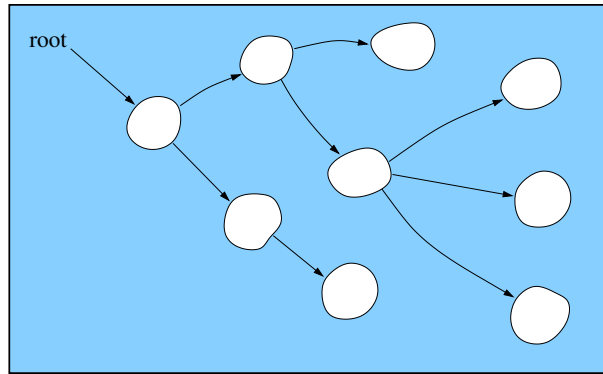
I stället för att lagra posterna i en sammanhängande minnesarea som för arrayer kan man låta dem peka ut varandra dvs. en post kan innehålla *adressen till* nästa post.



Fördelen med detta är att det går att länka in nya element var som helst utan att redan lagrade element behöver flyttas. En nackdel är att index-operationen kräver $\Theta(n)$ operationer eftersom vi måste börja från början och följa pekare tills vi kommer till elementet med sökt index.

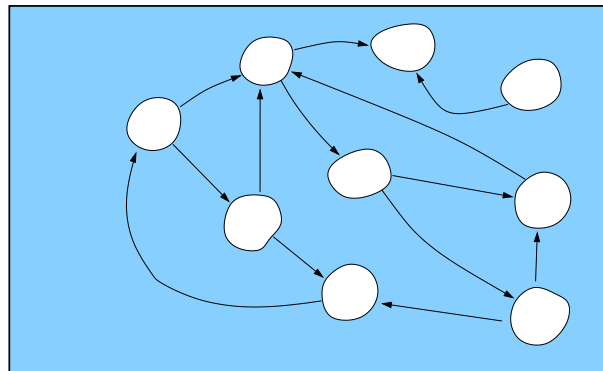
Träd

I länkade listor har varje post (högst) en efterföljare. Om man tillåter att en post har flera efterföljare får vi *trädstruktur*:



Grafer

Om en post kan ha flera relationer både bakåt och framåt och även cirkulärt kallar vi strukturen för en graf:



Hashtabell

En hashtabell är ett sätt att implementera en avbildning från en mängd av nycklar till en mängd av värden så att till varje element i nyckelmängden hör exakt ett element i värdemängden.

Datastrukturer i Python

Den fundamentala datastrukturen i Python är *listan* (`list`). Listan är implementerad som en array av referenser till posterna vilket medger snabb indexering ($\Theta(1)$). Eftersom listor kan utvidgas är arrayen *dynamisk* dvs. den växer vid behov.

Datatypen *tuple* (`tuple`) är lik en lista men eftersom den är oföränderlig (det går inte att ändra eller lägga till referenser) blir de ibland effektivare.

Datatypen *lexikon* (`dict`) representerar par av nycklar och värden. Denna är implementerad med hashteknik.

Många paket och moduler definierar flera datastrukturer. Exempelvis definierar paketet NumPy *multidimensionella arrayer* och paketet Pandas så kallade *dataframes*.

2 Länkade listor

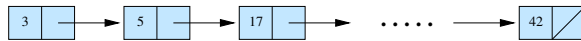
Pythons inbyggda, arraybaserade, listor är oerhört kraftfulla och oftast det bästa alternativet för allmänna listor. Det finns ändå skäl att studera hur man kan göra länkade listor. Det är ett enkelt sätt att illustrera tekniker att hantera alla möjliga dynamiska strukturer — inte bara linjära listor.

Dessutom är det en bra övning i rekursivt tänkande som man behöver tillämpa i mer komplicerade strukturer som träd och grafer.

Vi ska exemplifiera tekniken med listor med heltal som data i noderna. Listorna ska hållas sorterade så att data-delarna kommer i stigande ordning.

Att vi använder heltal som data är ingen begränsning — algoritmerna är generella. Det enda som är viktigt är att elementen kan jämföras med mindre än/större än.

En (enkel-)länkad lista med heltal kan åskådliggöras så här:



Vi representerar de enskilda elementen i listan med objekt ur klassen `Node`:

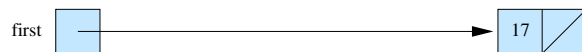
```
1 class Node:
2     def __init__(self, data, succ):
3         self.data = data
4         self.succ = succ
```

(Här och nu bekymrar vi oss inte med att förse instansvariablerna "skyddsmekanismer", d.v.s. att inleda namnen för interna variabler i klassen med `_`.)

För att hålla reda på listan behöver man en referens till första elementet.

Här följer en illustration av vilken effekt tre kodrader har:

```
first = Node(17, None)
```



```
first = Node(5, first)
```



```
first = Node(3, first)
```

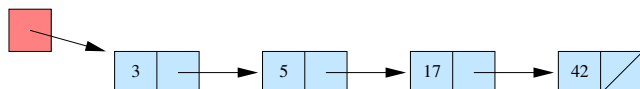


Nu kan vi iterera över listan och, till exempel, summera de lagrade talen:

Koden kommer producera raden
Sum: 25

```
1 summa = 0
2 f = first
3 while f:
4     summa += f.data
5     f = f.succ
6 print('Sum: ', summa)
```

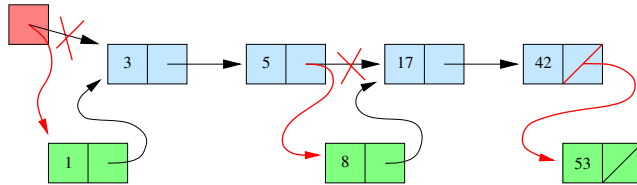
Antag nu att vi vill skriva kod som lägger in ett nytt värde i en sorterad lista så att sorteringen bibehålls.



Olika fall:

1. I en från början helt tom lista.
2. Först i listan. T.ex. inlägg av **1**
3. Sist i listan. T.ex. inlägg av **53**
4. Inuti listan. T.ex. inlägg av **8**

Det gäller att finna den pekare som skall ändras dvs. antingen pekaren till den första noden eller noden som ska komma *omedelbart framför* den nya noden:



Antag att värdet vi vill lägga in finns i variabeln `x`. Koden kan då skrivas:

```

1 if first == None or x <= first.data: # case 1 and 2
2     first = Node(x, first)
3 else:
4     f = first
5     while f.succ != None and x > f.succ.data:
6         f = f.succ
7     f.succ = Node(x, f.succ) # case 3 and 4
8

```

I det fall värdet redan finns lagras vi det nya värdet framför det gamla värdet. Koden blir aningen mer komplicerad om man vill lagra det nya värdet efter det gamla värdet. (Om det bara är heltal som lagras känns det tämligen oviktigt vilken ordning lika tal kommer i *men* om det är objekt med flera egenskaper så kan inläggningsordningen vara viktig.)

Det är alltså ett specialfall om det nya värdet ska placeras först i listan antingen beroende på att den är tom eller på att det nya värdet är mindre än det första elementet. Då är det ju variabeln `first` som ska ändras.

Om det nya värdet inte ska ligga första använder vi en `while`-loop för att hitta den nod som ska komma *närmast framför* den nya noden.

Kom ihåg att ordningen i testet är viktig — man måste först veta att det finns en nod innan man kan titta på dess innehåll!

En klass för listan

Det är lämpligt att förpacka den länkade listan med pekare och funktioner i en egen klass. Skiss:

```
1 class LinkedList:
2     class Node:
3         def __init__(self, data, succ):
4             self.data = data
5             self.succ = succ
6
7     def __init__(self):
8         self.first = None
9
10    def print(self):    # Unnecessary but serves as a simple example
11        pass
12
13    def insert(self, data):
14        pass
15
```

Klassen `Node` är samma som tidigare men den är nu placerad inuti klassen — den har ingen användning utanför `LinkedList`.

Klassen `LinkedList` har endast en instansvariabel: den första noden (eller, egentligen, en referens till första noden).

Från början sätts den till `None` dvs. listan är tom.

Inlägg i listan

Vi förpackar koden för inläggning som tidigare beskrevs i en metod:

```
1 def insert(self, x):
2     if self.first is None or x <= self.first.data:
3         self.first = LinkedList.Node(x, self.first)
4     else:
5         f = self.first
6         while f.succ and x > f.succ.data:
7             f = f.succ
8         f.succ = LinkedList.Node(x, f.succ)
```

Eftersom klassen `Node` är ett attribut i klassen så måste vi referera den med `LinkedList.Node`. I den första versionen skrev vi `f.succ != None` men här skriver vi `f.succ` som första villkor i loopen på rad 6. I ett villkor tolkas `None` som `False` så detta är bara ett kortare skrivsätt för samma sak. Det är bara säkert om de andra värden än `None` som kan förekomma kommer att tolkas som `True`. I det här fallet lagrar vi bara instanser av `Node` och det kommer att gälla.

En enkel print-metod

Som exempel på en enklare metod än `insert` skriver vi en metod för utskrift av listor. Den är egentligen onödig eftersom utskrifter skulle kunna göras med hjälp av metoden `__str__` men den tjänar som ett enkelt exempel.

Första försök:

```
1 def print(self):
2     f = self.first
3     while f:
4         print(f.data)
5         f = f.succ
```

Detta ger ett värde per rad vilket antagligen inte är det sättet man vill se listan.

Med lite mer ansträngning kan vi se till att alla värden kommer på samma rad med komma mellan och hela listan omgiven av parenteser:

```
1 def print(self):
2     print('(', end='')
3     f = self.first
4     while f:
5         print(f.data, end='')
6         f = f.succ
7         if f:
8             print(', ', end='')
9     print(')')
10
```

Koden som vi hittills gått igenom finns i den nedladdade `linked_list.py`.

Övning 1: Metoden `length`

Skriv en metod `length(self)` som, med hjälp av en `while`-loop, beräknar och returnerar antalet noder i listan. Den ursprungliga listan får inte förstöras.

Övning 2: Metoden `remove_last`

Skriv en metod som tar bort den sista noden ur listan. Metoden ska returnera värdet i den borttagna noden.

Om listan är tom ska ett `ValueError` genereras. (`ValueError` är en standardundantagsklass i Python.)

Övning 3: Metoden `remove`

Skriv en metod `remove(self, x)` som tar bort den första noden som innehåller värdet `x` som data. Om metoden hittar en sådan nod och tar bort den ska den returnera `True` och annars `False`.

Skriv metoden på ett iterativt sätt (med en loop). Vi kommer senare att ta upp hur man kan göra det rekursivt. Se till att metoden fungerar även om det är det första eller sista elementet.

3 Rekursiva listmetoder

Som nämndes inledningsvis passar länkade strukturer bra för att hanteras med rekursiva metoder. Det beror på att en lista kan definieras rekursivt på detta sätt:

En *lista* är
antingen
tom (består av 0 element)
eller
består av *ett element följt av en lista*.

Vi ska nu skriva metoden `length` rekursivt i stället för iterativt som i övningen högre upp. Man kan göra detta på två sätt.

Metod 1: Med en hjälp-metod i `Node`-klassen

Vi lägger in en metod i `nodklassen` som talar om hur lång den lista är som börjar med den noden:

```
1 class Node:
2     ...
3     def length(self):
4         if self.succ is None:
5             return 1
6         else:
7             return 1 + self.succ.length()
8     ...
```

Således, om den här noden inte har någon efterföljare så är längden 1, annars är den 1 *plus* längden av listan som börjar med efterföljaren.

I klassen `LinkedList` lägger vi metoden som ska användas utifrån:

```
1 def length(self):
2     if self.first is None:
3         return 0
4     else:
5         return self.first.length()
```

Vi måste ju börja med att kontrollera att det överhuvudtaget finns några noder i listan.

Metod 2: Med en hjälpmetod i `LinkedList`-klassen

I det andra sättet gör vi inget i `Node`-klassen utan skriver en intern metod `_length(self, f)` i `LinkedList`-klassen som returnerar hur många noder det finns i listan som börjar med `f` och låter den externt anropbara funktionen använda den:

```
1 def length(self):
2     return self._length(self.first)
3
4 def _length(self, f):
5     if f is None:
6         return 0
7     else:
8         return 1 + self._length(f.succ)
```

Man kan också skriva hjälpmetoden som en lokal funktion, inuti `length`.

```
1 def length(self):
2
3     def _length(f):
4         if f is None:
5             return 0
6         else:
7             return 1 + _length(f.succ)
8
9     return _length(self.first)
```

Eftersom den funktionen inte längre är en metod i klassen, tar den inte in `self` som parameter och anropas också utan `self`. först.

Vilket sätt man föredrar är en smaksak. Eftersom metod 2 ofta ger färre specialfall blir koden kortare, särskilt när man kommer till träd. Dessa metoder kan dock vara svårare att förstå.

Rekursiv `insert` med metod 1

Här ska vi skriva en `insert` i `Node`-klassen. När vi är i en nod går det inte att ändra på pekaren *till* noden utan bara på det som finns i noden. Precis som i den iterativa varianten måste vi alltså stanna på den som ska komma omedelbart framför den nya noden. Det betyder att vi bara ska gå vidare i rekursionen om den påföljande noden säkert ska vara framför den nya noden. Det ger följande kod i `Node`-objektet:

```
1 def insert(self, x):
2     if self.succ is None or x <= self.succ.data:
3         self.succ = LinkedList.Node(x, self.succ)
4     else:
5         self.succ.insert(x)
```

På rad 3 skapar vi alltså den nya noden.

Metoden i huvudklassen måste först kontrollera att den nya noden inte ska på första plats dvs. den ska vara säker på att det finns minst ett element i listan och att detta ska komma framför den nya noden.

Det ger följande kod i `LinkedList`:

```

1 def insert(self, x):
2     if self.first is None or x <= self.first.data:
3         self.first = LinkedList.Node(x, self.first)
4     else:
5         self.first.insert(x)

```

Lite tråkigt att behöva upprepa i princip samma kod. ...

Rekursiv insert med metod 2

I metod 2 placerar vi en hjälpmetod i huvudklassen. Hjälpmetoden får en referens till första noden i listan och ska returnera första noden i den modifierade listan.

```

1 def insert(self, x):
2     self.first = self._insert(x, self.first)
3
4 def _insert(self, x, f):
5     if f is None or x <= f.data:
6         return LinkedList.Node(x, f)    # Return the new node
7     else:
8         f.succ = self._insert(x, f.succ)
9         return f                        # Return the same node we got in

```

Observera hur hjälpmetoden `_insert` anropas på rad 2! Den nya noden kan ju komma först i listan.

Hjälpmetoden fungerar alltså så här:

Den får en referens till första noden i en lista och returnerar första noden i den modifierade listan. Om den nya noden kom först i listan är det den som returneras, men om den hamnade längre bort så returneras den "gamla" första noden.

Det som är trevligt med sättet är att det egentligen inte blir något specialfall för tomma listan eller att nya noden ska in först.

Följande övning ska lösas med rekursion i stället för iteration. Fortsätt att arbeta med den kod du laddade ner tidigare.

Övning 4: Metoden `to_list`.

Skriv en rekursiv metod `to_list(self)` som returnerar en standardlista med värdena i den länkade listan i samma ordning.

3.1 Iteratorer och generatorer för klassen `LinkedList`

För att behandla alla element i många av de vanliga strukturerna (listor, tupler, strängar, filer ...) kan man använda en `for`-sats. Man säger att dessa strukturer är *itererbara* (eng. *iterable*). För att en struktur ska vara itererbar måste det finnas en metod `__iter__` som returnerar en *iterator*.

En iterator är ett objekt som håller reda på ett *aktuellt element*. Klassen ska alltid innehålla en specifik metod, `__next__`. I vårt fall ska klassen både vara itererbar och en iterator.

En iterator för objekt ur klassen `LinkedList` skulle kunna se ut så här:

```
1 class LinkedListIterator:
2     def __init__(self, lst):
3         self.current = lst.first
4
5     def __iter__(self):
6         return self
7
8     def __next__(self):
9         if self.current:
10            res = self.current.data
11            self.current = self.current.succ
12            return res
13        else:
14            raise StopIteration
```

Konstruktorn (metoden `__init__`) tar emot (en referens till) ett `LinkedList`-objekt och sparar det i instansvariabeln `current`, metoden `__next__` returnerar värdet i det aktuella elementet och stegar fram till nästa element i listan. Om sista elementet har passerats så kastas undantaget `StopIteration`. Metoden `__iter__` returnerar bara sig själv.

Med denna klass kan man t.ex. skriva:

```
1 it = LinkedListIterator(lst)
2 for x in it:
3     print(x)
```

Pythons `for`-sats använder alltså precis de här metoderna. `__iter__` anropas först. Sedan anropas `__next__` upprepade gånger på det objekt som returnerades av `__iter__`.

Man kan också iterera över listan med en `while`-sats som utnyttjar `__next__` i `LinkedListIterator`:

```
1 it = LinkedListIterator(lst)
2 while True:
3     try:
4         print(next(it))
5     except StopIteration:
6         break
```

Det här kan ge en bild av vad `for`-satsen egentligen gör bakom kulisserna.

Observera att `next(it)` är ett trevligare sätt att skriva `it.__next__()`! Om du vill prova denna iterator kan du ladda ner [linked_list_iterator.py](#) (frivilligt).

I stället för att göra en separat iterator-klass kan man förse klassen `LinkedList` med metoderna `__iter__` och `__next__`.

Metoden `__iter__` får då uppgiften att skapa `current`-variabeln:

```
1 def __iter__(self):
2     self.current = self.first
3     return self
```

Observera att denna metod alltså lägger till en instansvariabel i klassen!

Metoden `__next__` är helt oförändrad.

Nu kan man iterera över den länkade listan på samma sätt som man itererar över vanliga listor:

```
1 for x in lst:
2     print(x)
```

Det kan finnas nackdelar med den här lösningen. Vad tror du skulle hända om man försökte

iterera över alla par:

```
1 for x in lst:
2     for y in lst:
3         print(x, y)
```

Det går att åstadkomma detta ännu enklare genom att skriva `__iter__` som en *generator*:

```
1 def __iter__(self):
2     current = self.first
3     while current:
4         yield current.data
5         current = current.succ
```

Python-satsen `yield` fungerar som `return` men nästa gång generatoren anropas kommer den fortsätta där den lämnade. Vi behöver inte skriva någon `__next__`-metod eller kasta något `StopIteration` — dessa skapas automatiskt.

Denna generator finns med i koden du laddade ner från början. Fungerar iterationen över par ovan med denna version? Varför?

Det finns många YouTube-videor om iteratorer och generatorer. Corey Schafer har till exempel gjort dessa om [iteratorer](#) respektive [generatorer](#).

3.2 Operatoröverlagring

För vanliga listor kan man använda operatoren `in` och t.ex. skriva uttryck som `if x in lista:`. Vi kan införa även detta i vår egen listklass genom att definiera metoden `__contains__`:

```
1 def __contains__(self, x):
2     for d in self:
3         if d == x:
4             return True
5         elif x < d:
6             return False # No point in searching more
7     return False
```

Denna kod finns med i klassen som du laddade ner från början.

Observera hur vi kan använda den egna generatoren för klassen i `for`-satsen.

På liknande sätt kan man definiera andra operatorer som t.ex. `<`, `<=`, `==`, ... genom att definiera metoderna `__lt__`, `__le__`, `__eq__`, ...

Corey Schaefer har också gjort en [YouTube-lektion](#) om magiska metoder och operatoröverlagring.

Övning 5: Metoden `__str__`.

Använd iteratorn eller generatoren för att skriva metoden `__str__(self)` som returnerar listan som en sträng med kommaseparerade värden och allt omgivet av parenteser.

Exempel på resultatsträngar: `()`, `(1)`, `(1, 2)`. Observera att resultatet måste ha *exakt* det utseendet dvs. med kommatecken följda av blanktecken *mellan* varje värde.

Övning 6: Metoden `copy`.

I koden du laddat ned ser `copy` ut som exempel till höger:
Denna kod skapar och returnerar en kopia av ett `LinkedList`-objekt.

```
1 def copy(self):  
2     result = LinkedList()  
3     for x in self:  
4         result.insert(x)  
5     return result
```

Vad blir tidskomplexiteten för denna metod?

Skriv om den för att ge den bättre komplexitet! Det är viktigt att kopian inte delar några nodobjekt med originalet!

Vilken komplexitet har din metod?

Övning 7: Lagra en klass `Person` i `LinkedList`

Antag att du har följande klass:

```
1 class Person:  
2     def __init__(self, name, pnr):  
3         self.name = name  
4         self.pnr = pnr
```

1. Skapa en tom lista `plist` i `main` tillsammans med ett personsobjekt `p`.
2. Lägg in personsobjektet i listan med `insert`-metoden i din version av klassen `LinkedList`.
Skriv ut `plist` med `print`. Detta bör fungera.

3. Skapa ett till personsobjekt `q` och försök lägga in det med `insert`.

Programmet kommer att avbrytas i `insert` på följande rad

```
if f is None or x <= f.data:
```

Felmeddelandet är

```
TypeError: '<=' not supported between instances of 'Person' and  
'Person'
```

Detta beror på att Python inte vet hur man ska jämföra två personsobjekt — `insert` förväntas sortera objekten i listan i storleksordning, men vet inte hur.

Om man vill kunna använda jämförelseoperatorer som `<`, `<=`, `=` ... mellan två objekt måste man implementera de magiska metoderna `__lt__`, `__le__`, `__eq__`, ...

Skriv de metoder som behövs för att det ska gå att lägga in instanser av `Person` i en `LinkedList`.

Du ska *inte* ändra något i `LinkedList`, utan enbart lägga till jämförelseoperatorer i klassen `Person`.

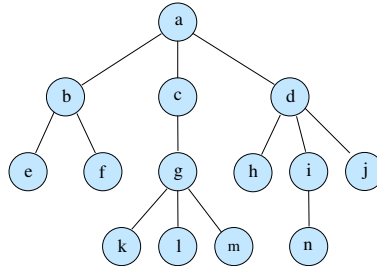
Du kan själv välja om `Person`-objekt ska sorteras efter namn eller personnummer (`pnr`).

4 Trädstrukturer

Definitioner och terminologi

I en lista har varje nod exakt en efterföljare (utom sista) och en föregångare (utom första).

Om vi tillåter att en nod har flera efterföljare erhåller vi en *trädstruktur*:



Terminologin hämtas från såväl skogen som familjen:

En *förälder* är en nod som har en eller flera efterföljare (*barn*).

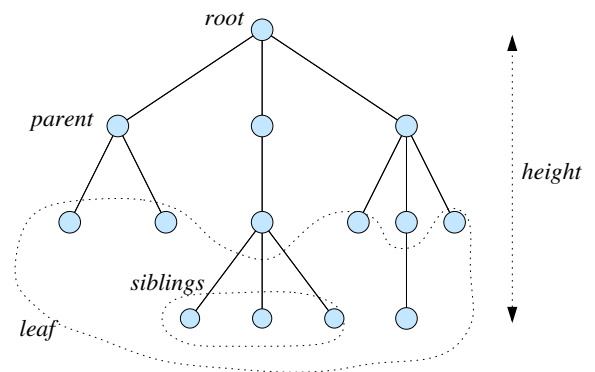
Ett *barn* är en nod som har en föregångare (*förälder*).

Barn med samma förälder kallas *syskon*.

Syskonen kommer i åldersordning med äldsta först (längst till vänster).

Ett *löv* är en nod utan barn.

Alla noder utom *roten* har en förälder.



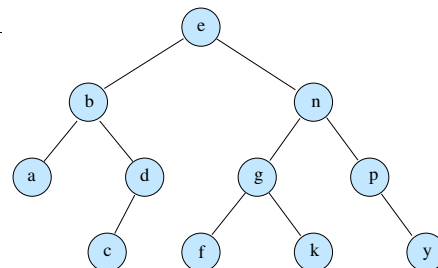
Trädets *höjd* är det största antalet noder på vägen mellan roten och ett löv. Höjden av ovanstående träd är alltså 4.

Anm: Ibland definieras höjden som antalet "bågar" mellan roten och ett löv. Med denna definition blir höjden av ovanstående träd 3. Det spelar ingen större roll vilken definition man använder bara man är konsekvent.

Ett *binärt träd* är en mängd noder som antingen är tom eller som består av tre delmängder:

1. en med exakt en nod — *roten*,
2. en delmängd kallad *vänster subträd* och
3. en delmängd kallad *höger subträd*.

De två subträden utgör i sin tur också binära träd.



Observera skillnaden mellan ett binärt träd och ett ordnat träd med högst två barn per nod:

1. Ett ordnat träd har minst en nod medan ett binärt träd kan vara tomt.
2. I ett binärt träd är det skillnad på att ha ett tomt högersubträd och ett tomt vänstersubträd — någon sådan skillnad finns inte för ordnade träd.

Operationer på träd

Det finns en mängd operationer man kan vilja utföra på träd beroende på tillämpning:

- *Traversering* dvs. gå igenom alla noder (i någon ordning).
- *Sökning* efter en nod med visst innehåll eller i en viss position.
- *Inlägg/uttag* av noder.
- *Sammanfogning* av flera träd.
- *Mätning* av höjd, bredd, antal noder, väglängder, balans ...

Här tar vi bara upp algoritmer för traverseringar.

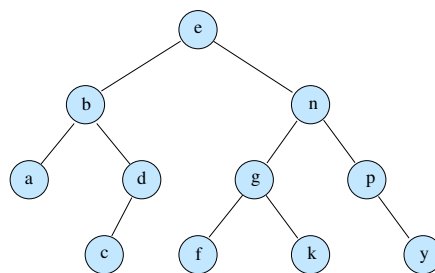
Övriga operationer diskuteras i kodningsexemplen och uppgifterna.

Traverseringar

Att traversera en struktur betyder att besöka alla element i någon viss ordning. För träd finns det huvudsakligen fyra tänkbara ordningar: *preordning*, *inordning*, *postordning* och *nivåordning*. (Oftast använder man dock engelsk terminologi för de tre första: *preorder*, *inorder* och *postorder*. För nivåordning talar man ibland om "bredden först".)

Preorder:

1. Visit the root
2. Visit the children in *preorder* (with the oldest first)



På exempelträdet blir ordningen: e b a d c n g f k p y

Postorder:

1. Visit the children in *postorder* (with the oldest first)
2. Visit the root

På exempelträdet blir ordningen: a c d b f k g y p n e

Inorder (for binary trees):

1. Visit the left subtree in *inorder*
2. Visit the root
3. Visit the right subtree in *inorder*

På exempelträdet blir ordningen: a b c d e f g k n p y

Level order:

1. Visit the root (level 0)
2. Visit all nodes on level 1
3. Visit all nodes on level 2
4. ...

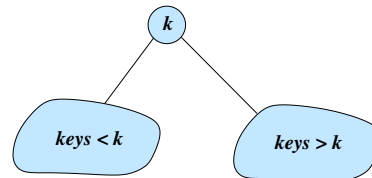
På exempelträdet blir ordningen: e b n a d g p c f k y

Binära sökträd

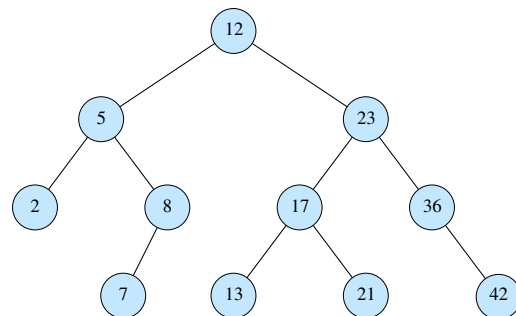
Binära *sökträd* är en viktig tillämpning av binära träd. I ett sådant träd har varje nod en *nyckel* som unikt definierar noden. En nyckel kan t.ex. vara ett personnummer, ett fordonsregistreringsnummer eller ett ord. Typiskt är alltså att det inte finns två eller flera noder med samma nyckel.

Nycklarna måste vidare ha en ordningsrelation definierad dvs. man ska kunna jämföra nycklarna i storlek.

Nyckeln i en nod ska vara större än alla nycklar i nodens vänstersubträd och mindre än alla nycklar i nodens högersubträd:

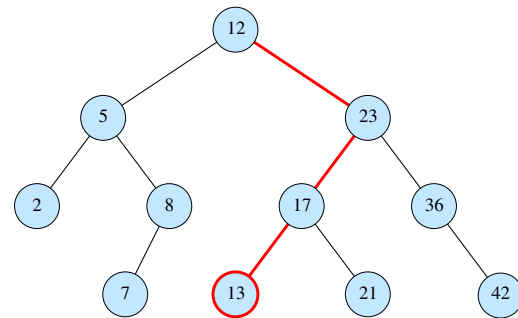


Exempel: Ett binärt sökträd med heltal som nycklar:

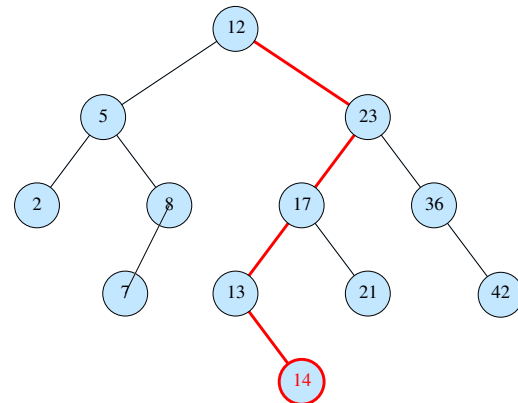


För att söka efter en nod med en given nyckel utgår man från roten. Ordningsrelationen kan sedan guida sökningen ner till den efterfrågade noden.

För att exempelvis hitta 13 kommer man gå till höger vid 12, till vänster vid 23 och 17 eftersom 13 är större än 12 men mindre än 23 och 17:



Man gör på samma sätt vid inlägg av nya nycklar: börjar från roten och låter noderna styra åt vilket håll man ska gå. Det finns alltid exakt en plats som en ny nyckel ska in på. Figuren visar inlägg av 14:



Slutsats: Arbetet att såväl söka som lägga in nycklar i ett binärt sökträd begränsas uppåt av *höjden* på trädets.

Representation och algoritmer för binära sökträd

Vi ska här bygga upp en struktur för att lagra poster i binära sökträd. I koden kallar vi de lagrade posterna för nycklar (**key**) och använder heltal som exempel, men alla typer av data som har ordningsrelationerna definierade fungerar bra. Vi tillåter inte att samma nyckel förekommer flera gånger.

Minimalt vill vi kunna skapa träd, lägga in nya poster, söka viss post och skriva ut trädets innehåll.

```

1 class BST:
2
3     class Node:
4         def __init__(self, key):
5             self.key = key
6             self.left = None
7             self.right = None
8
9     def __init__(self):
10         """Create an empty tree"""
11         self.root = None
12
13     def insert(self, key):
14         """Insert a new key in the tree"""
15         pass
16
17     def contains(self, k):
18         """Check if k is in the tree"""
19         pass
20
21     def print(self):
22         """Print the contents of the tree"""
23         pass
24

```

Metoden `print` finns med från början för att enkelt kunna skriva ut träd men den kommer senare att ersättas av den mera generella `__str__`-metoden.

Trädet består alltså av nodobjekt där varje nod innehåller en nyckel samt referenser till vänster respektive höger subträd.

Trädobjektet har bara instansvariabeln `root` som initieras till `None` dvs. trädet är från början tomt.

Vi börjar med att titta på metoden `contains`:

Observera att sökningen styrs av relationen mellan det sökta värdet och innehållet i noden.

Iterationen pågår så länge `n` inte är `None` och nyckeln inte finns i `n`.

Om `n` inte är `None` när loopens slut har värdet hittats.

```

1 def contains(self, k):
2     n = self.root
3     while n and n.key != k:
4         if k < n.key:
5             n = n.left
6         else:
7             n = n.right
8     return n is not None

```

Vi har skrivit denna metod iterativt. I många fall är det dock betydligt lättare att skriva trädmetoder rekursivt som bl.a. `print`-metoden är ett exempel på.

Metoden skriver ut trädet på en rad med noderna i storleksordning.

```

1 def print(self):
2     self._print(self.root)
3
4 def _print(self, r):
5     if r:
6         self._print(r.left)
7         print(r.key, end=' ')
8         self._print(r.right)
9

```

Detta mönster med en hjälpmetod som får en referens till en nod att arbeta med och en huvudmetod som bara sätter igång hjälpmetoden genom att skicka in rotnoden kommer vi använda i många trädmetoder.

Till skillnad från `print` går metoden `insert` lätt att skriva iterativt, men vi gör det ändå rekursivt.

```
1  def insert(self, key):
2      self.root = self._insert(self.root, key)
3
4  def _insert(self, r, key):
5      if r is None:
6          return self.Node(key)
7      elif key < r.key:
8          r.left = self._insert(r.left, key)
9      elif key > r.key:
10         r.right = self._insert(r.right, key)
11     else:
12         pass # Already there
13     return r
```

Den här tekniken visade vi också som "metod 2" i avsnittet om länkade listor dvs. hjälpmetoden *returnerar* en referens till rotnoden i det modifierade subträdet. Det som är trevligt med denna teknik är att vi inte behöver skilja på fallen helt tomt träd, nya noden är ett vänsterbarn och nya noden är ett högerbarn.

Studera koden och se till att du förstår den!

Vi visar ytterligare en metod som beräknar antalet noder. Den använder samma rekursiva teknik med hjälpmetod som en del av ovanstående metoder.

```
1  def size(self):
2      return self._size(self.root)
3
4  def _size(self, r):
5      if r is None:
6          return 0
7      else:
8          return 1 + self._size(r.left) + self._size(r.right)
```

En generator

I klassen `LinkedList` presenterade vi en generator för att successivt kunna iterera över noderna. Nyckelordet där var `yield`. Vi ska titta på en sådan för vår BST-klass som går över noderna i inorder vilket innebär storleksordning.

Vi börjar med att skriva en generator för nodklassen:

```

1  class Node:
2      ... # Same as before
3      def __iter__(self):
4          if self.left:           # If there is a node to the left
5              for n in self.left: # use its generator
6                  yield n.key     # to return node references
7          yield self              # Then return ourself
8          if self.right:         # Then the nodes to the right
9              for n in self.right:
10                 yield n
11

```

Denna är rekursiv i den mening att en generator i en nod utnyttjar en generator i andra noder.

Observera att vi måste kontrollera att det finns en nod innan vi försöker använda dess generator.

Detta går att skriva lite mer kompakt genom konstruktionen `yield from`:

```

1  class Node:
2      ...
3      def __iter__(self):
4          if self.left:
5              yield from self.left
6          yield self.key
7          if self.right:
8              yield from self.right
9

```

Nu kan vi enkelt göra en generator för trädet:

```

1  def __iter__(self):
2      if self.root:
3          yield from self.root

```

Radera noder i binära sökträd

Det är lite mer komplicerat att ta bort en nod ur ett binärt sökträd jämfört med övriga operationer vi har behandlat. Om nyckeln hör till en nod utan barn, eller med bara ett barn, är det inga större problem. Då påminner det mycket om hur man gör för att ta bort noder ur länkade listor. Däremot blir det mer komplicerat när man ska ta bort en nod med två barn, eftersom det inte blir självklart hur de ska förenas.

Det vanligaste angreppssättet blir att behålla nodobjektet, men byta ut dess innehåll (nyckel) med nyckeln från en annan nod i trädet. Eftersom trädets sökegenskap (lägre nycklar till vänster, högre till höger) måste bevaras finns det bara två nycklar som kan vara den rätta ersättaren till den nyckel som tas bort: den *största* bland de lägre eller den *minsta* bland de högre. Båda dessa nycklar har egenskapen att de högst kan ha ett barn. Annars skulle något av barnen i sin tur vara större/mindre. Det spelar ur korrekthetsperspektiv ingen roll vilken av dessa två du väljer.

Pseudokod för hela processen:

```

1 def remove(self, key):
2     self.root = self._remove(self.root, key)
3
4 def _remove(self, r, k):
5     if r is None:
6         return None
7     elif k < r.key:
8         r.left = # left subtree with k removed
9     elif k > r.key:
10        r.right = # right subtree with k removed
11    else: # This is the key to be removed
12        if r.left is None: # Easy case
13            return r.right
14        elif r.right is None: # Also easy case
15            return r.left
16        else: # This is the tricky case.
17            # Find the smallest key in the right subtree
18            # Put that key in this node
19            # Remove that key from the right subtree
20    return r # Remember this! It applies to some of the cases above

```

Observera att vi har separata rekursioner på rad 8, 10 och 19.

Övning 8: Metoden `contains`

Skriv `contains`-metoden med rekursion i stället för iteration.

Observera att `contains` kontrolleras i `main`.

Det finns ingen testfil för denna metod, men du kan först skriva lämpliga tester som fungerar med den ursprungliga `contains`-metoden. Använd sedan dessa test för din egen version av metoden.

Övning 9: Beräkna höjd

Skriv en metod `height` som returnerar höjden av trädet. Det tomma trädet har höjd 0, trädet med 1 nod har höjd 1, trädet med 2 noder har höjd 2 medan trädet med 3 noder har höjd 2 eller 3 beroende på inläggningsordning.

Övning 10: Metoden `__str__`

Generatorkoden finns med i den nedladdade filen. Använd denna för att skriva en `__str__`-metod. Elementen i strängen ska *separeras* av kommatecken följt av ett blanktecken och strängen ska omges av '`<`' och '`>`'. Exempel på resultat: '`<>`', '`<1>`' och '`<1, 2>`'.

Övning 11: Metoden `to_list`

Skriv metoden `to_list` som skapar och returnerar en standardlista med värdena från trädet. Vad är komplexiteten?

Övning 12: Metoden `to_LinkedList`

Skriv metoden `to_LinkedList` som skapar och returnerar en `LinkedList` med värdena från trädet. Vad är komplexiteten?

Övning 13: Ta bort en nyckel

Utgå från den givna pseudokoden och skriv klart `remove`.

Övning 14: Komplexitet för sökning

I ett binärt sökträd med n noder och höjd h , vad blir komplexiteten i värsta fallet?

- Värsta fallet, lyckad sökning
- Värsta fallet, misslyckad sökning

I senare frivilligt övningsmaterial kommer det att finnas mer teori om komplexitet för binära träd.